

Computerwoche-Inhalte schnell finden und dank KI die richtigen Antworten erhalten? Testen Sie unsere neue hybride Suchfunktion! **Jetzt ausprobieren!**

# COMPUTERWOCHE

# COMPUTERWOCHE

Home • Business-Software

---



von **Serdar Yegulalp**  
Senior Writer

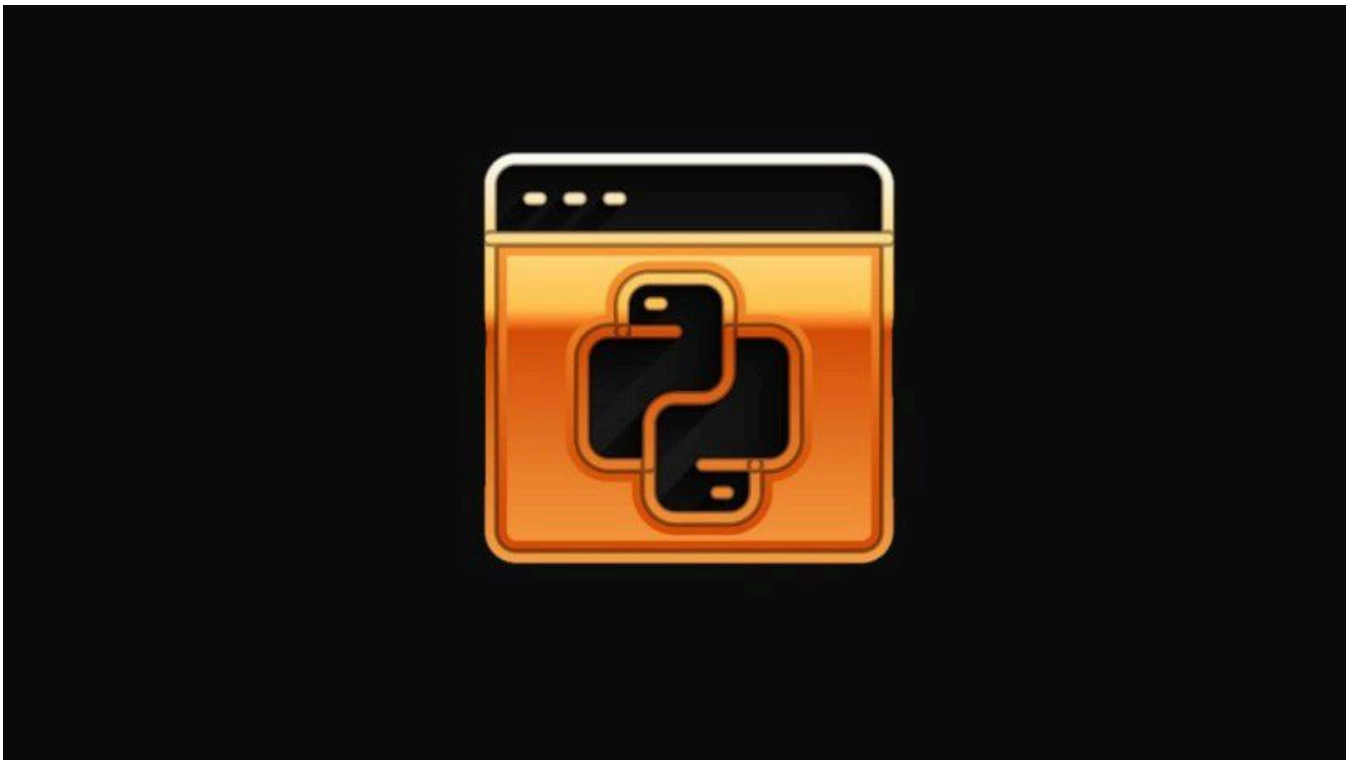
## PyScript-Einführung: Python im Browser ausführen

How-To

18. Juni 2026 • 6 Minuten

PyScript ermöglicht Entwicklern, Python-Skripte direkt im Browser auszuführen. Eine Einführung.

---



PyScript bringt Python-Skripte direkt in den Browser – und erschließt Interaktionsmöglichkeiten zwischen Code und Webseite.

Foto: VectorV | shutterstock.com

**PyScript** [<https://pyscript.net/>] wurde von einem Team um Peter Wang bei **Anaconda Inc.** [<https://www.anaconda.com/about-us>] entwickelt und im April 2022 veröffentlicht. Die Technologie macht die Python Runtime als Skriptsprache in **WebAssembly** [<https://www.computerwoche.de/article/2811214/vom-aufstieg-eines-browser-tools.html>]-fähigen Browsern verfügbar. Dabei zielt das Framework darauf ab, eine vollständige Browser-Umgebung bereitzustellen, um Python als Web-Skriptsprache auszuführen.

Zwar handelt es sich bei **PyScript** [<https://pyscript.net/>] um ein experimentelles Projekt, das (noch) nicht für Produktionszwecke geeignet ist – allerdings lohnt es sich für Interessierte, mit den bislang verfügbaren Komponenten erste Python-trifft-JavaScript-im-Browser-Experimente zu realisieren.

In diesem Artikel lesen Sie, wie das geht.

## Mit PyScript programmieren

Im Kern besteht PyScript aus einem einzigen **JavaScript** [<https://www.computerwoche.de/article/2821289/7-javascript-projekte-die-sie-kennen-sollten.html>]-Include, das zu einer Webseite hinzugefügt wird. Dieser Include lädt die Basis-Runtime von PyScript und fügt automatisch Support für benutzerdefinierte Tags hinzu. Im Folgenden ein einfaches Beispiel für ein Hello-World-Projekt in PyScript:

```
<!DOCTYPE html>  
<html>
```

```
<head>
<link rel="stylesheet"
href="https://pyscript.net/releases/2023.11.2/core.css" />
<script type="module"
src="https://pyscript.net/releases/2023.11.2/core.js">
</script>
</head>
<body>
<script type="py" terminal>
from pyscript import display
display("Hello World!")
print("Hello terminal!")
</script>
</body>
</html>
```

Das `script` -Tag im `head` des Dokuments lädt die Kernfunktionalität von PyScript. Das `.css` -Stylesheet ist optional, aber praktisch, weil es den Benutzern beim Laden der Seite unter anderem mitteilt, was gerade passiert.

Über das Attribut `type="py"` wird Python-Code in den `script` -Tag inkludiert. Dabei sollten Sie beachten, den Code gemäß den Python-Namenskonventionen zu formatieren. Ansonsten besteht die Gefahr, dass er nicht richtig ausgeführt werden kann. Insbesondere wenn Sie einen Editor verwenden, der HTML automatisch umformatiert, ist das relevant. Der Inhalt des `script` -Blocks könnte verfälscht werden. Sie können auch auf ein `.py` -File referieren, was sich einfacher gestalten kann, als das Script inline zu inkludieren.

Sobald die PyScript-Komponenten fertig geladen sind, wird der Python-Code evaluiert. Dabei können Sie wählen, ob der Output an das DOM (mit `pyscript.display` ) oder ein eingebettetes Terminal gesendet werden soll. Wenn Sie Letzteres verwenden wollen, müssen Sie `terminal` als Attribut in das `script` -Tag aufnehmen (dazu später mehr).

Wenn das Skript in den Tags nach `stdout` schreibt (wie bei einem `print` -Statement), können Sie bestimmen, wo der Output auf der Seite angezeigt werden soll, indem Sie eine Property für `output` definieren. Im obenstehenden Beispiel wird `stdout` für das Skript an das `div` mit der ID von `"out"` weitergeleitet.

Wenn Sie diesen Code in einer Datei speichern und erstmals im Webbrowser öffnen, sehen Sie zunächst einen "Lade"-Indikator. Der Browser ruft zunächst die PyScript-Laufzeit ab und richtet diese ein. Zukünftige Ladevorgänge sollten dank Zwischenspeicherung flotter ablaufen. Anschließend sollte Hello world zweifach auf der Seite erscheinen – einmal oben in HTML und einmal in einem schwarzen Fenster, dem Embedded Terminal.

# Computerwoche Smart Answers [Mehr erfahren](#)

## Verwandte Themen erkunden

- [Welche anderen Programmiersprachen lassen sich für den Browser in WebAssembly kompilieren?](#)
- [Wie ermöglicht Pyodide die bidirektionale Kommunikation zwischen meinem Python-Code und JavaScript?](#)
- [Wie nutze ich reine Python-Pakete aus PyPI erfolgreich in der Pyodide-Laufzeitumgebung?](#)
- [Wie vereinfacht Hyperscript alltägliche Frontend-Aufgaben direkt in meiner bestehenden Browser-Umgebung?](#)

## Eine Frage stellen

---



FRAGEN

## Von Standard-Bibliotheken importieren

Skripte, die ausschließlich Python-Builtins verwenden, sind nur bedingt nützlich. Die Standardbibliothek von Python lässt sich in PyScript auf identische Weise wie bei "normalem" Python nutzen: Ein `import` -Befehl genügt. Entsprechend sollten Importe aus der Standardbibliothek einfach funktionieren.

Wenn Sie den obigen Skriptblock ändern wollten, um die aktuelle Zeit anzuzeigen, würden Sie das wie von Python gewohnt erledigen:

```
import datetime
print ("Current date and time:",
datetime.datetime.now().strftime("%Y/%m/%d %H:%M:%S"))
```

## Bibliotheken über PyPI verwenden

Mit PyScript können Sie Projektkonfigurationen, die von PyPI installiert werden sollen – einschließlich aller Drittanbieter-Packages – über eine `.toml` – oder `.json` -Datei im

Projektverzeichnis angeben. Im Folgenden betrachten wir, wie das mit `.toml` funktioniert. Um die Projektkonfigurationsdatei zu verwenden, müssen Sie die `config` -Direktive in Ihr Skript-Tag einfügen:

```
<script type="py" src="main.py" config="pyscript.toml">
```

In der Datei `pyscript.toml` sind alle benötigten Packages aufgeführt:

```
packages = ["package", "another-package"]
```

Dabei gilt es zu beachten, dass nicht alle Packages von PyPI installiert und ausgeführt werden können. Die meisten "reinen" Python-Pakete (etwa `humanize`) sollten problemlos laufen, genauso wie `numpy`, `pandas`, `bokeh` oder `matplotlib`. Anders sieht es bei Packages aus, die Netzwerkzugriff benötigen oder mit plattformspezifischen Elementen wie GUIs arbeiten. Sie werden höchstwahrscheinlich nicht funktionieren.

## Lokal importieren

Ein weiteres gängiges Szenario: Andere Python-Skripte importieren, die sich im selben Verzeichnisbaum wie Ihre Webseite befinden. Importe zu verwenden, macht es einfacher mehr Python-Logik von der Webseite selbst auszulagern. Denn dort ist sie mit Ihrer Präsentation "vermischt", was es schwierig machen könnte, damit zu arbeiten.

Normalerweise nutzt Python andere, vorhandene `.py` -Files im Dateisystem, um anzuzeigen, was es importieren kann. PyScript arbeitet nicht auf diese Weise. Deswegen müssen Sie angeben, welche Dateien als importierbare Module verfügbar sein sollen.

Dazu listen Sie die entsprechenden URLs in der Konfigurationsdatei Ihrer Anwendung in einem `[files]`-Block auf und definieren, wie Sie dem emulierten Dateisystem von PyScript zugeordnet werden sollen. Zum Beispiel:

```
[files]
"/module.py" = "../libs/module.py"
"https://mydata.com/data.csv" = "../data.csv"
```

Jede Datei, die über die URL auf der linken Seite zugänglich ist, wird dem emulierten Dateisystem des Python-Interpreters über den Pfad auf der rechten Seite zur Verfügung gestellt. In diesem Fall ist die Datei, die Sie sehen würden, wenn Sie `/module.py` aufrufen würden, für Python als `libs.module` verfügbar. Zudem ist die Datei unter der URL `https://mydata.com/data.csv` im emulierten, aktuellen Arbeitsverzeichnis verfügbar.

## Das In-Browser-Terminal

Python-Benutzer sollten mit REPL, der Konsolenschnittstelle zur Python-Laufzeitumgebung, vertraut sein. In PyScript können Sie diese in ein Live-Terminal in den Browser einbetten – oder einfach nur den Konsolen-Output Ihres Python-Programms.

Um ein Terminal einzubetten, verwenden Sie ein Skript-Tag, das `terminal` als Attribut ausweist:

```
<script type="py" terminal>print("hello world")</script>
```

Für Interaktivität müssen Sie das Attribut `worker` verwenden:

```
<script type="py" terminal worker>
name = input("What is your name? ")
print(f"Hello, {name}")
</script>
```

Der `worker` führt Ihr Programm in einem Web Worker aus, bei dem es sich im Wesentlichen um einen Unterprozess handelt. Zu beachten ist an dieser Stelle, dass Web Worker nicht für eine lokal geladene HTML-Datei verwendet werden können – Sie müssen sie von einem Webserver laden, der bestimmte Header bereitstellt. Wie das im Detail funktioniert, lesen Sie in der PyScript-Dokumentation [<https://pyscript.github.io/docs/2023.12.1/user-guide/workers/>].

In einem PyScript-Terminal haben Sie größtenteils dieselben Möglichkeiten, wie in einer herkömmlichen Konsole – inklusive Farbgebung und Unicode.

## Mit DOM und JavaScript interagieren

Weil PyScript auf Browser-Technologie basiert, verfügt es über Mechanismen zur DOM-Interaktion [<https://pyscript.github.io/docs/2023.12.1/user-guide/dom/>]. Wenn Sie etwa den Wert eines Eingabefeldes auf einer Webseite abrufen und in Ihrem Python-Code verwenden möchten, gehen Sie folgendermaßen vor:

```
from pyscript import window, document
inputbox = document.querySelector("#my-input")
print("Value of input:", inputbox.value)
```

PyScript enthält auch ein Modul namens `pydom`, das ermöglicht, Objekte auf der Seite dynamisch zu erstellen:

```
from pyweb import pydom
new_div = pydom.create("div")
```

```
new_div.content = "Hello World"
```

Das erzeugt ein neues `div`-Element auf der Seite und befüllt es mit Text. Mit der `pydom`-Bibliothek sind die meisten Anpassungen, die mit DOM in JavaScript möglich sind, durchführbar. (fm)

Dieser Artikel ist im Original [<https://www.infoworld.com/article/2335480/intro-to-pyscript-run-python-in-your-web-browser.html>] bei unserer Schwesterpublikation Infoworld.com erschienen.

Business-Software[<https://www.computerwoche.de/enterprise-applications/>]

NEWSLETTER

# First Look

Von KI bis Business Software: Mit unserem täglich versandten First Look Newsletter bleiben Sie auf dem Laufenden.

☐ Mit der Übermittlung Ihrer Daten erklären Sie sich mit unserer DATENSCHUTZERKLÄRUNG [<https://foundryco.com/privacy-policy/>] einverstanden.

**JETZT ANMELDEN**

Über Uns



Rechtliches



Mehr



Unser Netzwerk



© 2026 FoundryCo, Inc. All Rights Reserved.





